CT Porosity, Volume and Surface Analysis

Gohel Mahek Vipulkumar
Mehta Family School for Data Science and Artificial Intelligence, IIT Roorkee

Under the supervision of
Prof. Mayank Goswami
Department of Physics, IIT Roorkee
and
Mr. Ankit Singh Bhadauriya

Abstract

Porosity is a key quality indicator for cast, printed and sintered parts, and X-ray computed tomography (CT) makes it measurable in three dimensions. We describe a small, dependency-light C++17/OpenCV pipeline for specimens that appear as a bright circular ring in each slice and whose pores appear as dark gaps in that ring. The pipeline (i) detects the ring automatically in every slice with a robust least-squares circle fit and declares slices without a ring as having no region of interest, (ii) thresholds dark pixels inside a thin band around the fitted circle, (iii) labels two-dimensional blobs and links blobs that overlap in consecutive slices into three-dimensional pores using union–find, and (iv) reports, for every pore, its volume, equivalent diameter, surface area, Wadell sphericity, principal axes, aspect ratio, fill ratio and angular position on the ring, together with a CSV file of per-pore features. On a synthetic 60-slice stack with five pores of known size, of which twelve slices carry no ring, all five pores were recovered with exactly the true voxel counts when the ring was wider than the pores, and the elongated pore was flagged as irregular. When the ring was only 3.5 px wide, the same pores were found but their volumes were underestimated by 36–81%, which we identify as the main limitation of measuring pores only inside the ring band. On one real slice the pipeline fitted the ring and reported no pores, as expected for that slice. We have not yet validated the method against labelled real data, and we state the resulting limits explicitly. The code and the synthetic-data generator are provided.

Keywords: X-ray computed tomography; porosity; pore segmentation; connected components; union–find; sphericity; OpenCV

1 Introduction

Pores are small voids inside a solid. Their number, size and shape influence strength, fatigue life and leak tightness, so porosity is routinely quantified in additively manufactured, cast and sintered components [1]. X-ray computed tomography reconstructs a specimen as a stack of cross-section images (*slices*); a pore is dark in such images because X-rays pass through a void more easily than through the material. Quantifying pores in three dimensions therefore requires (a) deciding where in each slice the specimen is, (b) deciding which pixels are pores, (c) recognising that a pore seen in several consecutive slices is one object, and (d) measuring that object.

This paper documents a compact pipeline that solves these four tasks for a specific but common geometry: the specimen cross-section is a thin bright ring, and pores are dark spots on the ring. Our contributions are:

- an automatic per-slice region-of-interest (ROI) detector based on Otsu thresholding and a robust Kasa circle fit, with explicit validity checks so that slices without a ring are recognised and skipped;

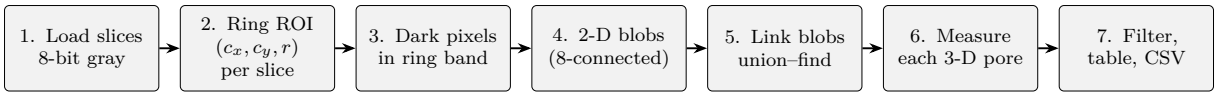


Figure 1: Pipeline overview. Each stage has one input and one output, so a wrong result can be traced to a single stage.

- a ring-band pore threshold relative to the local ring brightness, with an edge-shrink that suppresses false one-pixel pores on thin, discretised rings;
- a three-dimensional linking step based on overlap in consecutive slices and union–find, followed by per-pore measurements including a surface-area estimator calibrated for round objects;
- an honest evaluation on a synthetic stack with known ground truth, an ablation on ring width and edge shrink, and one real slice, together with a list of the method's limitations.

2 Related work

General-purpose image-analysis platforms such as Fiji [3] and its bone-analysis extension BoneJ [4] provide thresholding, connected-component labelling and shape measurements, and are widely used for tomographic data; reviews of X-ray imaging and analysis of pore structure are given in [2, 1]. Our pipeline does not aim to replace these tools. It targets a narrower case, ring-shaped sections, and offers a single-file, scriptable program in which the ROI is found automatically in every slice and every tuning value is a command-line option.

The individual components are standard. Otsu's method selects a global threshold by maximising between-class variance [5]. The algebraic circle fit of Kasa [6] minimises an algebraic rather than a geometric error and has a closed-form solution. The median absolute deviation (MAD), scaled by 1.4826, is a robust estimate of spread [7]. Union–find with path compression [8, 9] merges equivalence classes efficiently. Sphericity follows Wadell [**?**]. Surface area of digitised objects is known to be biased when counted from voxel faces, and better estimators exist [**?**]; we use a simple calibration that is exact in expectation only for round objects (Section 3.4).

3 Method

Figure 1 summarises the pipeline. Let a stack consist of N_z slices $I_z(x, y)$, $z = 0, \ldots, N_z - 1$, converted to 8-bit grayscale (16-bit data are scaled by 255/65535, colour data converted with the standard luminance weights). The voxel size is $(\Delta x, \Delta y, \Delta z)$.

3.1 Automatic ring detection (ROI)

For each slice we first reject slices whose maximum intensity is below 30. Otherwise we compute Otsu's threshold [5] and collect the coordinates $\{(x_i, y_i)\}_{i=1}^{n}$ of the bright pixels; at least 200 are required. A circle with centre (c_x, c_y) and radius r is fitted by the algebraic least-squares method of Kasa [6]: writing $u_i = x_i - \bar{x}$, $v_i = y_i - \bar{y}$, we minimise

$$\sum_i \left(u_i^2 + v_i^2 + D u_i + E v_i + F\right)^2, \tag{1}$$

which is linear in (D, E, F); then $c_x = \bar{x} - D/2$, $c_y = \bar{y} - E/2$ and $r^2 = D^2/4 + E^2/4 - F$.

Stray bright pixels bias the fit, so we refit six times. After each fit the residual of every point is $e_i = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2} - r$. With $m = \operatorname{median} |e_i|$ over the current inliers, the inlier tolerance is

$$\tau = \max\!\left(3 \text{ px},\ 3 \cdot 1.4826\, m\right), \tag{2}$$

Algorithm 1: Linking 2-D blobs into 3-D pores

Input : pore masks M_z and ROI flags v_z, $z = 0, \ldots, N_z - 1$
Output : set of 3-D pores

```
for z ← 0 to N_z − 1 do
    if not v_z then mark "previous slice invalid"; continue;
    label 8-connected components of M_z to obtain L_z; create one node per label;
    if v_{z−1} then
        foreach pixel (x, y) with L_z(x, y) > 0 and L_{z−1}(x, y) > 0 do
            UNITE(node_z(L_z(x, y)), node_{z−1}(L_{z−1}(x, y)));
group nodes by FIND; discard groups with fewer than 4 voxels or 2 slices;
```

and the next fit uses the points with $|e_i| < \tau$.

A fitted circle is accepted only if all of the following hold: the inliers are at least 60% of the bright pixels; the inliers occupy at least 70% of 72 equal angular sectors; the radius is between 20 px and $0.6 \min(W, H)$; and the ring thickness $r_{\text{out}} - r_{\text{in}}$ is at most $0.3\, r$. Here r_{in} and r_{out} are the 2nd and 98th percentiles of the inlier radii, and the ring brightness B is the median intensity of the inliers. A slice that fails any check has *no ROI*; it is skipped and the reason is logged. This is how slices without a ring are handled without user input.

3.2 Pore pixels inside the ring band

Let $\rho(x, y) = \sqrt{(x - c_x)^2 + (y - c_y)^2}$. With an edge shrink s (default 0.6 px) the search band is $[r_{\text{in}} + s,\ r_{\text{out}} - s]$ (the shrink is capped at $0.4(r_{\text{out}} - r_{\text{in}})$). A pixel is a pore pixel if

$$r_{\text{in}} + s \le \rho(x, y) \le r_{\text{out}} - s \quad \text{and} \quad I_z(x, y) < f\, B, \tag{3}$$

where f is the pore fraction (default 0.5). Because the threshold is relative to the brightness B of the ring in the same slice, it adapts to slice-to-slice intensity changes. An optional 3×3 morphological opening removes speckle (off by default).

The edge shrink is needed because a thin ring drawn on a pixel grid has ragged edges; some pixels inside the nominal band are simply dark and would otherwise be reported as one-pixel pores (Section 5.4).

3.3 Two-dimensional labelling and three-dimensional linking

Pore pixels in each slice are grouped into 8-connected components (2-D blobs). Each blob becomes a node of a union–find structure [8, 9]. Two blobs in consecutive slices $z - 1$ and z are united if they share at least one (x, y) pixel. Linking is performed only if both slices have an ROI, so a slice without a ring breaks the chain. All nodes with the same root form one three-dimensional pore (Algorithm 1). A pore is kept if it has at least 4 voxels and spans at least 2 slices; for a single-slice input the second requirement is lowered to 1 and the result is a two-dimensional analysis.

3.4 Per-pore measurements

Let a pore consist of N voxels.

Volume and equivalent diameter. $V = N\, \Delta x\, \Delta y\, \Delta z$ and $d_{\text{eq}} = (6V/\pi)^{1/3}$, the diameter of the sphere of equal volume.

Surface area. Let F_x, F_y, F_z be the numbers of voxel faces with normal along x, y, z that have no neighbouring pore voxel. The raw staircase area is

$$A_{\mathrm{f}} = F_x\,\Delta y\,\Delta z + F_y\,\Delta x\,\Delta z + F_z\,\Delta x\,\Delta y. \tag{4}$$

A smooth surface element with unit normal $\mathbf{n}$ is approximated by axis-aligned faces whose total area is $(|n_x| + |n_y| + |n_z|)$ times the element's area. For a sphere, $\langle|n_x|\rangle = \langle|n_y|\rangle = \langle|n_z|\rangle = 1/2$, so the staircase area exceeds the true area by a factor $3/2$ on average. We therefore estimate

$$A = \tfrac{2}{3} A_{\mathrm{f}}. \tag{5}$$

This estimator is calibrated for round objects; for other shapes (for example a cube, where the raw count is already exact) it is biased, and more accurate local-configuration estimators exist [**?**].

Sphericity. Following Wadell [**?**], the sphericity is the ratio of the surface area of the equal-volume sphere to the surface area of the pore,

$$\Psi = \min\!\left(1,\ \frac{\pi^{1/3}(6V)^{2/3}}{A}\right). \tag{6}$$

The clamp at 1 is needed because A is an estimate; as a consequence a perfect voxel cube is reported with $\Psi = 1$ (Section 6).

Principal axes and aspect ratio. Let $\mathbf{C}$ be the covariance matrix of the voxel centres in physical units, to which we add $\mathrm{diag}(\Delta x^2, \Delta y^2, \Delta z^2)/12$ (the variance of a voxel itself) so that one-voxel-thick pores remain well conditioned. Its eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$ give the full axis lengths of an equivalent solid ellipsoid, $a = 2\sqrt{5\lambda_1}$, $b = 2\sqrt{5\lambda_2}$, $c = 2\sqrt{5\lambda_3}$, because a solid ellipsoid with semi-axis s has variance $s^2/5$ along that axis. The aspect ratio is $c/a = \sqrt{\lambda_3/\lambda_1}$.

Fill ratio, angle and flag. Let $R_{\max}$ be the largest distance from the centroid to a voxel centre and $R_s = R_{\max} + \frac{1}{2}(\Delta x \Delta y \Delta z)^{1/3}$. The fill ratio is $V/(\frac{4}{3}\pi R_s^3)$. The angular position of the pore on the ring is $\mathrm{atan2}(\bar{y} - c_y,\ \bar{x} - c_x)$ in degrees, using the mean ring centre over the slices the pore occupies. A pore is flagged *spherical* if $\Psi \geq 0.70$ and $c/a \geq 0.50$ (both thresholds are options).

4 Implementation

The program is a single C++17 source file using OpenCV [**?**] for image input, Otsu thresholding, connected components and the eigen-decomposition, and `std::filesystem` for listing slices. Input is one or more stacks, each a folder of slices (sorted by file name) or a multi-page TIFF. Output is a console table, an optional CSV file with one row per pore (a ready-made feature vector for downstream machine learning), and optional overlay images (fitted ring in green, kept pores in red and numbered). All tuning numbers live in one parameter structure and are command-line options (Table 1). On the 60-slice synthetic stack of 1000×1000 pixels used below, a single run took about 0.85 s with warm file cache on one core of an Intel Xeon @ 2.8 GHz (the first, cold-cache run took 5.8 s), with a peak memory of about 120 MB.

5 Experiments

5.1 Data

Real slice. We had one real slice (`Top_1_037.tif`, 1000×1000 pixels, 8-bit values stored in three identical channels). It shows a bright ring on a black background (Figure 2). Non-zero

Table 1: Default parameters (all are command-line options except where noted).

Parameter	Default	Meaning
`-pore-frac` f	0.5	pore if gray $< f \cdot B$
`-band-shrink` s	0.6 px	ring edge ignored on both sides
`-open`	0	3×3 opening iterations
`-min-voxels`	4	smallest pore kept
`-min-slices`	2	fewest slices a pore spans
`-min-sphericity`	0.70	spherical flag
`-min-aspect`	0.50	spherical flag (c/a)
`-ring-tol`	3 px	minimum inlier tolerance
`-min-coverage`	0.70	fraction of 72 sectors with ring
`-dx -dy -dz`	1	voxel size
(fixed) refit rounds, band percentiles	6; 2/98	ring fit



(a) Real slice (cropped) (b) Zoom of the ring top: 3–4 px wide

Figure 2: The real slice `Top_1_037.tif`. The ring is thin and its edges are ragged at the pixel level.

pixels have gray values between 120 and 226. The ring is only about 3–4 pixels wide. The slice shows no pore: the ring is continuous around the whole circumference. We have no pore annotations for this or any other real slice.

Synthetic stack. Because no labelled real data were available, we generated a synthetic stack with known ground truth (`make_synthetic_stack.py`): 60 slices of 1000×1000 pixels, a ring of radius 186 px centred at $(498.6, 496.6)$ with mean intensity 190 and Gaussian noise ($\sigma = 6$), and five ellipsoidal pores placed on the ring at angles $30°, 100°, 200°, 280°, 330°$. Four pores are (nearly) spherical with semi-axes 2.5 to 4.5 px; the pore at 280° is elongated along z (semi-axes $3 \times 3 \times 9$). Slices 0–5 and 54–59 contain only noise and no ring. Pore voxels are set to a dark value (≈ 5). We used two ring widths: 10 px (wider than the pores) and 3.5 px (similar to the real slice). The ground truth is the number of voxels in each ellipsoid mask. The generator and the detector share the assumptions of a bright ring, dark pores and sharp edges, so this experiment tests the internal consistency and the limits of the pipeline, not its performance on real scans.

Table 2: Synthetic stack, ring 10 px wide: ground truth versus detection.

Angle	True voxels	Found	d_{eq} (px)	Ψ	c/a	Flag
30°	263	263	7.95	0.999	0.93	spherical
100°	111	111	5.96	1.000	0.89	spherical
200°	337	337	8.63	0.987	0.88	spherical
280° (3:3:9)	340	340	8.66	0.920	0.33	irregular
330°	64	64	4.96	1.000	0.91	spherical

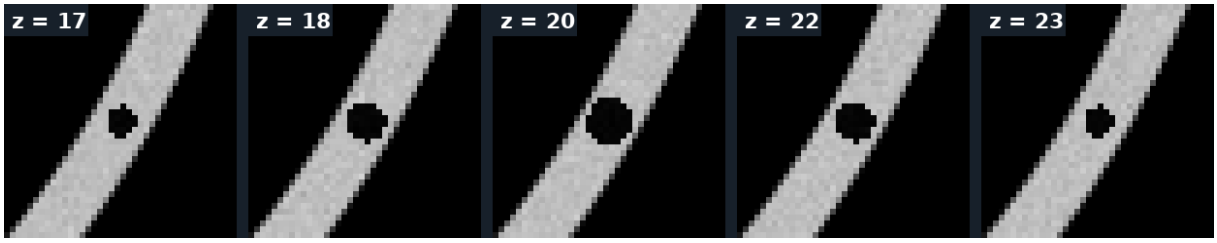


Figure 3: One pore of the synthetic stack in slices $z = 17, 18, 20, 22, 23$. The dark disc grows, peaks and shrinks while its position barely changes, so blobs in neighbouring slices overlap; this overlap is the linking criterion.

5.2 Ring detection on the real slice

The fitted ring has centre (499.4, 496.9), radius 186.2 px, band [184.9, 187.4] px and median brightness 186, so the pore threshold is $T = 0.5 \cdot 186 = 93$. These values agree with a direct computation from the non-zero pixels (radial range 184.0–188.3 px). With the default edge shrink the pipeline reports no pore on this slice.

5.3 Synthetic stack, wide ring

Of the 60 slices, 48 received an ROI and the 12 ring-less slices were correctly skipped. All five pores were found, at the correct angles, with voxel counts equal to the ground truth (Table 2). The elongated pore has $c/a = 0.33$ against 0.88–0.93 for the round ones and is flagged irregular, although its sphericity (0.92) alone would not separate it, which motivates the additional aspect-ratio criterion. Figure 3 shows one pore through consecutive slices, and Figure 4 an overlay.

5.4 Ablations

Edge shrink. On the real slice, before introducing the edge shrink, the pore threshold inside the unshrunk band produced 24 candidate regions, all of a single pixel. With the shrink (default 0.6 px) there were none. Such regions are artefacts of the thin, discretised ring, not pores; in a stack they would also persist at the same (x, y) in consecutive slices and could be linked into spurious columns.

Ring width. Table 3 repeats the synthetic experiment with a ring 3.5 px wide. All five pores were still detected, but their volumes were 36–81% lower than the truth, because the program only sees pore pixels inside the ring band and the pores are larger than the band is wide.

6 Discussion and limitations

- **Pores larger than the ring are clipped.** Pores are measured only inside the ring band. If the ring is narrower than the pore, volume, surface and shape are biased (Table 3).

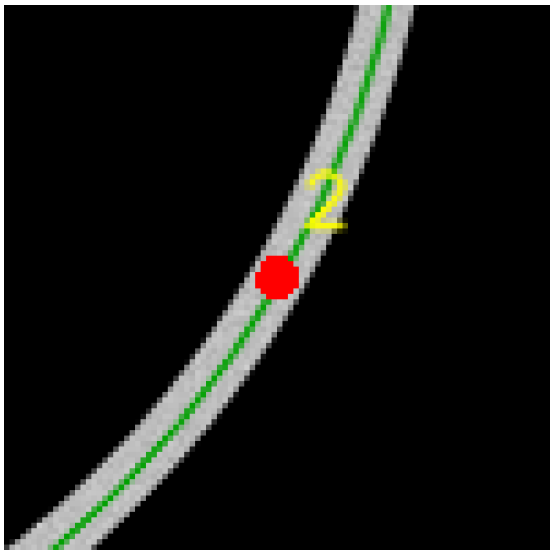


Figure 4: Debug overlay (slice $z = 20$, cropped): fitted ring in green, detected pore in red with its identifier.

Table 3: Voxel counts: ground truth versus detection for two ring widths.

Angle	True	Found, 10 px ring	Found, 3.5 px ring	Underestimate (3.5 px)
30°	263	263	106	60%
100°	111	111	60	46%
200°	337	337	138	59%
280°	340	340	171	50%
330°	64	64	41	36%

Whether this matters depends on the pore size relative to the ring width in the real data, which we could not check; a pore-size distribution from real annotated scans is needed.

- **No validation on labelled real data.** The only real slice has no pore, so true-positive and false-negative rates on real scans are unknown. The synthetic test shares the generator's assumptions with the detector.
- **Surface and sphericity are estimates.** The factor 2/3 is exact in expectation only for round objects; a voxel cube is reported with $\Psi = 1$ after clamping, and pores of only a few voxels give unreliable values. The aspect ratio and fill ratio should be read together with Ψ.
- **Global assumptions.** The ring is assumed bright and circular, pores darker than fB, and a real pore continuous across neighbouring slices. A slice without a ring breaks the link chain, so a pore straddling such a slice is split.
- **Simple linking.** Overlap-based linking with 8-connectivity in-plane can merge touching pores or split a pore whose blobs do not overlap between slices; 26-connected labelling or watershed splitting would be natural extensions.
- **Possible improvements.** Denoising before thresholding, a sector-wise adaptive threshold, smoothing the ring centre and radius along z, and a learned pore classifier trained on the exported CSV features.

7 Conclusion

We presented a compact pipeline that detects a ring-shaped ROI in each CT slice, finds dark pores inside the ring, links them in three dimensions with union–find and reports volume, size, surface, sphericity and shape per pore. On a synthetic stack the pipeline recovered all pores exactly when the ring was wider than the pores, handled slices without a ring automatically, and exposed a clear limitation for thin rings. The next step is validation against annotated real scans.

A Command-line example and CSV schema

```
g++ -O2 -std=c++17 pore_analyzer.cpp -o pore_analyzer \
    $(pkg-config --cflags --libs opencv4)
./pore_analyzer stackA stackB --dx 0.5 --dy 0.5 --dz 1.0 --unit um \
    --csv pores.csv --debug overlays
```

The CSV has one row per pore with the columns

```
stack, pore_id, z_start, z_end, voxels, volume, eq_diameter,
surface_area, sphericity, axis_a, axis_b, axis_c,
aspect_c_over_a, fill_ratio, cx, cy, cz, angle_deg, is_spherical
```

References

[1] E. Maire and P. J. Withers. Quantitative X-ray tomography. *International Materials Reviews*, 59(1):1–43, 2014.

[2] D. Wildenschild and A. P. Sheppard. X-ray imaging and analysis techniques for quantifying pore-scale structure and processes in subsurface porous medium systems. *Advances in Water Resources*, 51:217–246, 2013.

[3] J. Schindelin et al. Fiji: an open-source platform for biological-image analysis. *Nature Methods*, 9:676–682, 2012.

[4] M. Doube et al. BoneJ: free and extensible bone image analysis in ImageJ. *Bone*, 47(6):1076–1079, 2010.

[5] N. Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979.

[6] I. Kåsa. A circle fitting procedure and its error analysis. *IEEE Transactions on Instrumentation and Measurement*, 25(1):8–14, 1976.

[7] P. J. Rousseeuw and C. Croux. Alternatives to the median absolute deviation. *Journal of the American Statistical Association*, 88(424):1273–1283, 1993.

[8] B. A. Galler and M. J. Fischer. An improved equivalence algorithm. *Communications of the ACM*, 7(5):301–303, 1964.

[9] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.